# Enhancing System Security: LLM-Driven Defense against Prompt Injection Vulnerabilities

Oleksandr Muliarevych
*Department of Electronic Computing Machines*
*Lviv Polytechnic National University*
Lviv, Ukraine
oleksandr.v.muliarevych@lpnu.ua

***Abstract*—This article examines cybersecurity vulnerabilities in systems utilizing Language Model Interfaces, focusing on the challenges of building secure systems. It provides an overview of current interfaces and their associated risks. A key contribution is the design of a prompt analysis and injection detection subsystem, which assesses input relevance and security. The integration of an additional Filter level in the system safeguards against prompt injection attacks by pre-processing user requests and post-processing responses. This level includes a Prompt Analyzer for input wrapping, and an Attack Validator module using the LLM to classify prompts. The study evaluates various prompt injection attacks across different system setups, including core GPT-3 and GPT-4 models without additional security layers, a Fuzzy Search method, and systems using an Attack Validator based on GPT-3 and GPT-4 models.**

***Keywords—language model, prompt injection attack, cybersecurity for LLMs, ChatGPT, security layer integration***

## I. Introduction

Large language models (LLMs) are advanced computational systems designed to understand and generate natural language, playing a pivotal role in various applications. Their deployment in environments handling sensitive data necessitates stringent security measures to prevent unauthorized access, data breaches, and misuse. As LLMs become integral to critical infrastructure, the risks associated with their security increase, making robust cybersecurity essential. This article examines the security challenges of LLMs, including prompt injection attacks and the development of multi-layered cybersecurity frameworks. Additionally, it proposes a prompt analysis and injection detection subsystem to enhance LLM security, which is a so-called LLM-driven defense.

## II. Security in Systems with LLM

The significance of cybersecurity and the challenges developers face when building systems with LLM-based interfaces are evident [1]. This section classifies the primary issues to help understand and organize them. When discussing the security challenges of LLMs, it's essential to consider the broader cybersecurity frameworks and standards that address various risk levels [2]. A typical LLM-based system includes preprocessing, post-processing, filtering modules, an API/REST interface for internal and external communication, the LM itself, and a knowledge base containing user data. Monitoring is also crucial.

### *A. Security Framework Levels*

- Privacy Compliance: Addresses how client data is maintained, stored, and protected, with frameworks like GDPR [3] needing adaptation for LLMs.
- Data Security: Focuses on data access, encryption, and storage security, with standards like ISO 27000 [4] being relevant.
- Model Protection: Involves safeguarding the model from unauthorized changes, data poisoning, and adversarial attacks, with frameworks such as STRIDE or OCTAVE [5] being applicable.
- Model Integrity: Ensures secure communication between LLMs and other systems, with OAuth and OWASP [6] tools being useful.
- Filter Layer: Serves as an auxiliary level for ethical processing, detecting, and preventing prompt injection attacks.
- Monitoring: Involves continuous monitoring, anomaly detection, and incident response using SIEM [7] systems and frameworks like NIST's [8] Incident Response Guidelines.
- Standardization and Certification: Entails adapting industry standards like ISO/IEC 27001 [4] for systems using LLMs to establish consistent security.

During the design of a system with an interface based on an LLM application, we need to handle all described cybersecurity issues and protect prompt processing from prompt injection attacks. Let's describe the typical system with a set of functions to execute an LLM that is used for system interaction with the user like a chatbot. LLM is used for user request processing, and triggers different functions like "place an order", "create an application", "verify current order list", "suggest product from the marketplace", etc. To enable the supporting feature of LLM the set of files with guidance was added to the knowledge base, including company-sensitive data for higher quality results of responses (calculation formulas, margins, REST contracts for external API calls to other inner sub-systems, etc.).

To mitigate all the LLM-driven interface design risks it looks effective to use a Filter cybersecurity level. The conceptual model of the described system with protection from different types of prompt injection attacks is represented in Fig. 1. The structural schema illustrates a system utilizing a Language Model (LM) interface for user interaction, such as a chatbot. The colored modules limited by the dashed rectangle are related to cybersecurity filtering level implementation. Access to System Event Handler – this is the entry point where user interactions begin. It manages incoming requests and directs them through the authentication and authorization process. Authentication and Authorization – this module verifies the identity of the user and checks for the necessary permissions to proceed with the request. Caching

2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET) | 979-8-3315-2056-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/TCSET64720.2024.10755823

979-8-3315-2056-4/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Arizona State University. Downloaded on January 23,2026 at 15:59:11 UTC from IEEE Xplore. Restrictions apply.

Processor with Elasticsearch – this module before passing the prompt to the core LM checks the Elasticsearch cluster [9] to see if a similar prompt has already been processed, thereby reducing the workload on the LLM and speeding up response times, including the costs related to each prompt execution using LLM. Elasticsearch contains a database of previously asked questions and the LM's responses to facilitate quick retrieval of common inquiries and reduce redundant processing. Core LLM – the central LM that processes prompts and generates responses. It's connected to various subsystems that provide additional features or handle specific tasks. Shown Sub-System 1 to N represents various features or functions that can be triggered by the LM, such as placing an order or providing a product suggestion [10].

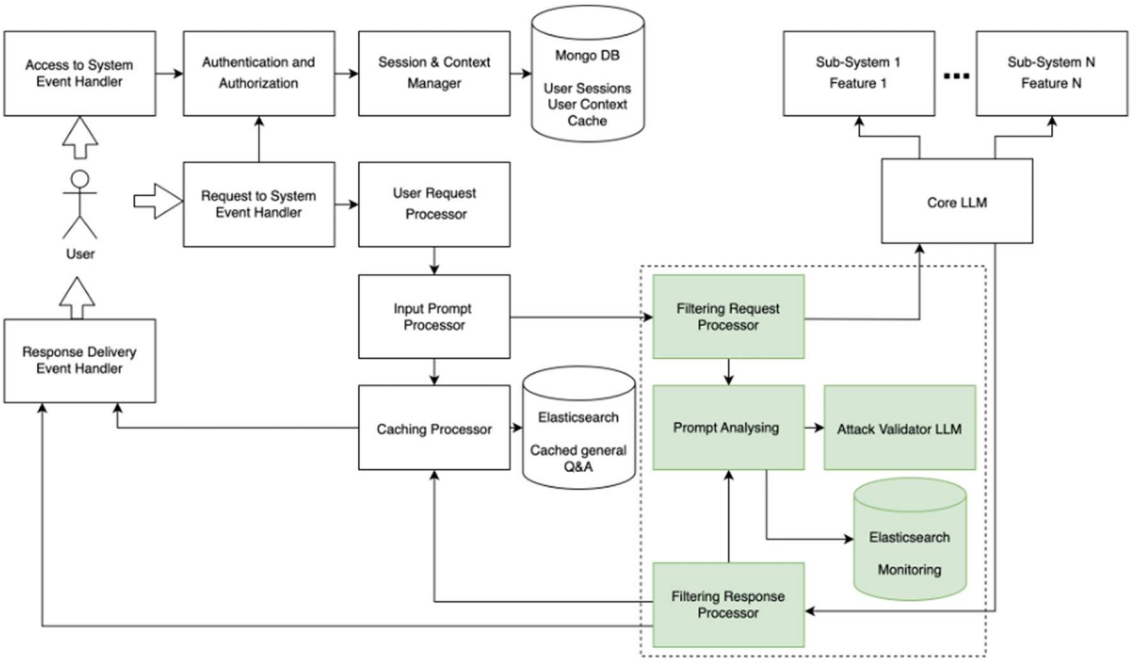


Fig. 1. Structural schema of System with implemented protection from prompt injection attacks.

User sessions and context persistence can be managed using NoSQL databases like MongoDB [10], which also allows caching frequent interactions to enhance system response times. The System Event Handler manages authenticated user requests, directing them to the appropriate processor. The User Request Processor interprets requests and determines the necessary action or response from the LM. Before reaching the core LM, the Filtering Request Processor analyzes prompts for potential risks like prompt injection attacks. A specialized helper LM, the Attack Validator LLM (tested with ChatGPT versions 3 and 4), further examines prompts for malicious intent. It categorizes prompts as either "confirmed" (indicating a potential attack) or "safe". Confirmed attacks prevent the request from reaching the core LM, with the user receiving a warning and a potential ban for repeated misuse.

## III. Experimental Scenarios

In this section, enhancing system security using an LLM-driven attack validator subsystem in different modes was investigated and its effectiveness was demonstrated. The Attack Validator LLM, implemented as an additional security layer, will be assessed for its capability to detect prompt injection attacks in systems utilizing Large Language Models (LLMs). The experimental design for the evaluation includes a diverse set of prompts reflecting real-world scenarios where LLMs might be compromised by various cybersecurity threats. The prompts used in these experiments represent a range of potential cybersecurity vulnerabilities. Each test run incorporates 800 prompts, consisting of 500 prompts designed to simulate specific types of attacks and 300 prompts representing regular, safe usage scenarios. The attack prompts are organized into five distinct categories: Model Memorization Probing, Architecture and Configuration Probing, Bias Probing, Unauthorized Access Probing, and Prompt-Chain Attacks, with each category containing 100 unique prompts. These prompts were carefully compiled from a variety of sources, including documented security incidents, literature on known vulnerabilities, and expertly created examples of system probing techniques. The test methodology involves evaluating the system's response to each type of prompt under various security configurations. This includes five separate test runs, each focusing on a different attack type, to ensure a comprehensive analysis across the spectrum of potential threats. In each test run, 100 attack prompts and an equivalent number of safe prompts are randomly processed through each security solution. This process is replicated across three distinct security configurations for performance comparison:

- A baseline setup without an additional security filtering layer, utilizing only the core GPT-3 LLM with basic capabilities from the default ChatGPT version 3, without specialized wrapping or additional security measures.
- A baseline setup without an additional security filtering layer, utilizing only the core GPT-4 LLM with basic capabilities from the default ChatGPT version 4, also without specialized wrapping or additional security measures.
- An advanced security configuration employing an Attack Validator based on a simple Fuzzy Search designed to detect common patterns associated with attack prompts.

Authorized licensed use limited to: Arizona State University. Downloaded on January 23,2026 at 15:59:11 UTC from IEEE Xplore. Restrictions apply.

- An advanced security configuration employing an Attack Validator based on a customized GPT-3 LLM that includes wrapped input prompt analyses.
- An advanced security configuration employing an Attack Validator based on a customized GPT-4 LLM with wrapped input prompt analyses.

For clarity, let's describe the test scenario, which was used per each system's configuration setup. Runs 1-5: 100 prompts specific to the Model Memorization Probing attack and 100 safe prompts are processed randomly. After each run, all caches are cleared, and the session is reset. Runs 6-10: 100 prompts specific to the Architecture and Configuration Probing attack and 100 safe prompts are processed randomly. After each run, all caches are cleared, and the session is reset. Runs 11-15: 100 prompts specific to the Bias Probing attack and 100 safe prompts are processed randomly. After each run, all caches are cleared, and the session is reset. Runs 16-20: 100 prompts specific to the Unauthorized Access Probing attack and 100 safe prompts are processed randomly. After each run, all caches are cleared, and the session is reset. Runs 21-25: 100 mini sets of prompts, with chains ranging from 2 to 5 in length (totaling 457 user inputs), specific to the Prompt-Chain attack, are processed randomly along with an equal number of safe prompts. After each run, all caches are cleared, and the session is reset. For each run, the number of True Positives (correctly identified attacks), True Negatives (correctly processed safe prompts), False Positives (safe prompts incorrectly flagged as attacks), and False Negatives (missed attacks) are recorded for each solution. This scenario is repeated five times, once for each system configuration, ensuring that each solution is tested against the same set of prompts. The results of these experiments are quantified using standard evaluation metrics such as accuracy, precision, recall, and the F1 score, enabling comparisons of their effectiveness across different scenarios. Table 1 presents statistics and evaluation metrics for Model Memorization Probing Attacks. Models specifically designed as attack detectors significantly outperform the default versions of GPT-3 and GPT-4, underscoring the importance of targeted security measures for this type of probing. The FuzzySearch (FS) validator, using a pattern vocabulary, shows potential for further improvement in precision and recall with increased training data. LLM-based GPT Validators demonstrate superior performance compared to other alternatives.

TABLE I. MODEL MEMORIZATION PROBING ATTACK MITIGATION

| System Setup Type | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| Default GPT-3 | 0,51 | 0,54 | 0,13 | 0,21 |
| Default GPT-4 | 0,59 | 0,84 | 0,21 | 0,34 |
| FS validator | 0,64 | 0,7 | 0,48 | 0,57 |
| GPT3 validator | 0,82 | 0,83 | 0,81 | 0,82 |
| GPT4 validator | 0,92 | 0,91 | 0,93 | 0,92 |

TABLE II. ARCHITECTURE AND CONFIGURATION PROBING ATTACK MITIGATION

| System Setup Type | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| Default GPT-3 | 0,48 | 0,39 | 0,07 | 0,12 |
| Default GPT-4 | 0,54 | 0,73 | 0,11 | 0,19 |
| FS validator | 0,54 | 0,58 | 0,28 | 0,38 |
| GPT3 validator | 0,8 | 0,81 | 0,78 | 0,79 |
| GPT4 validator | 0,86 | 0,87 | 0,84 | 0,85 |

Not employing any security layer proves to be highly ineffective, as this type of attack circumvents the default GPT models in nearly 80% of the attack prompts. Table 2 contains statistics and evaluation metrics per Architecture and Configuration Probing Attack. Compared to the Model Memorization Probing Attack in this case all approaches demonstrate a decrease in accuracy. Important to mention that Fuzzy Search solution has a higher rate of mistakes in the detection of safe and attack prompts correctly. It is caused by the complexity and variety of patterns applicable for such type of attack. Helper Validator models significantly outperform all other system setup types with a detection rate of more than 78%, GPT-4 again dominating with an 84% detection rate. Table 3 contains statistics and evaluation metrics per Bias Probing Attack. For this kind of attack, the Fuzzy Search solution is the worst option. All GPT models, defaults, and Helper Validator LLM based on GPT demonstrate a good level of accuracy, precision, and recall (more than 60% of attack detections). The best results have GPT-4, even without wrapping of input prompts and additional security layer, default GPT can detect more than 90% of attacks, likely due to its advanced understanding of nuanced language patterns. Table 4 contains statistics and evaluation metrics per Unauthorized Access Probing Attack. Similar picture to what we have with the Model Memorization Probing Attack stats, but the Fuzzy Search solution is closer by recall to GPT-4 and has a higher rate of mistakes in the detection of safe prompts correctly. Unfortunately, all default GPT versions do not detect role exchange attacks correctly. However, using the system with an additional Attack Validator GPT-4 model helped to overcome that issue, with a detection rate of more than 76%. Table 5 contains statistics and evaluation metrics per Prompt-Chain Attack. For this kind of attack, the Fuzzy Search shows the worst results. All default GPT models are able in some cases to detect attacks even from a single input. Utilization of GPT-4 demonstrates better results, which may be due to its more sophisticated understanding of complex prompt patterns.

TABLE III. BIAS PROBING ATTACK MITIGATION

| System Setup Type | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| Default GPT-3 | 0,75 | 0,85 | 0,61 | 0,71 |
| Default GPT-4 | 0,94 | 0,96 | 0,91 | 0,93 |
| FS validator | 0,55 | 0,58 | 0,31 | 0,4 |
| GPT3 validator | 0,82 | 0,88 | 0,73 | 0,8 |
| GPT4 validator | 0,96 | 0,95 | 0,97 | 0,96 |

TABLE IV. UNAUTHORIZED ACCESS ATTACK MITIGATION

| System Setup Type | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| Default GPT-3 | 0,56 | 0,68 | 0,23 | 0,34 |
| Default GPT-4 | 0,7 | 0,92 | 0,44 | 0,6 |
| FS validator | 0,61 | 0,67 | 0,41 | 0,51 |
| GPT3 validator | 0,7 | 0,77 | 0,58 | 0,65 |
| GPT4 validator | 0,83 | 0,88 | 0,76 | 0,82 |

TABLE V. PROMPT-CHAIN ATTACK MITIGATION

| System Setup Type | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|
| Default GPT-3 | 0,53 | 0,61 | 0,17 | 0,27 |
| Default GPT-4 | 0,62 | 0,87 | 0,27 | 0,41 |
| FS validator | 0,43 | 0,25 | 0,07 | 0,11 |
| GPT3 validator | 0,65 | 0,74 | 0,49 | 0,59 |
| GPT4 validator | 0,87 | 0,9 | 0,85 | 0,87 |

Authorized licensed use limited to: Arizona State University. Downloaded on January 23,2026 at 15:59:11 UTC from IEEE Xplore. Restrictions apply.

Wrapping and adding the context improve the rate of detection, in such a way Attack Validator of GPT-4 with high precision and accuracy can detect up to 85% of attacks. There is a trade-off between precision and recall in some cases, particularly with the fuzzy search method, which has lower precision but sometimes higher recall than the default GPT versions. This implies that while it may flag more true attacks, it also misclassifies safe prompts more often. Models that are specifically designed or trained to detect attacks (GPT-3 and GPT-4 Attack Validators) are more precise and have higher F1 Scores compared to general-use models (Default GPT-3 and GPT-4), underscoring the need for specialized security mechanisms. The progression from GPT-3 to GPT-4 shows a clear improvement in detecting attack patterns. This suggests that newer LLM versions are more adept at understanding and identifying potential security threats in text. The best setup of the system is using helper Attack Validator LLM based on GPT-4, with a detection attack rate from 76% to 97% depending on the type of prompt injection attack. Fig. 2 provides a graphical representation of the response times logged during testing different types of Attack Validator modules when processing input prompts. This figure illustrates both the average (avg) and maximum (max) time, measured in milliseconds (ms), taken by each setup to analyze a single prompt. The left bars represent the average time taken by each system, indicating typical performance under standard operational conditions. The red bars denote the maximum time recorded, reflecting how each module performs under the highest load or in the most complex scenarios with a big context window.

The data suggests that while Fuzzy Search is the fastest on average – 71 ms, the more sophisticated GPT-3 and GPT-4 LLM systems, despite taking longer (289 ms and 348 ms accordingly), provide a deeper level of prompt analysis. The subsequent analysis sheds light on the relative strengths and weaknesses of each approach, contributing valuable insights into the development of more secure LLM systems. The forthcoming results are instrumental in guiding future research and development in the field of AI and cybersecurity, highlighting the importance of adaptive and resilient defense mechanisms in the age of sophisticated language models. Simple methods like Fuzzy Search are outperformed by specialized AI models, highlighting the complexity and sophistication required for effective cybersecurity in the context of LLMs. These conclusions point towards a need for continuous advancement in LLM technology and specialized configurations to adequately protect against sophisticated cyber threats.

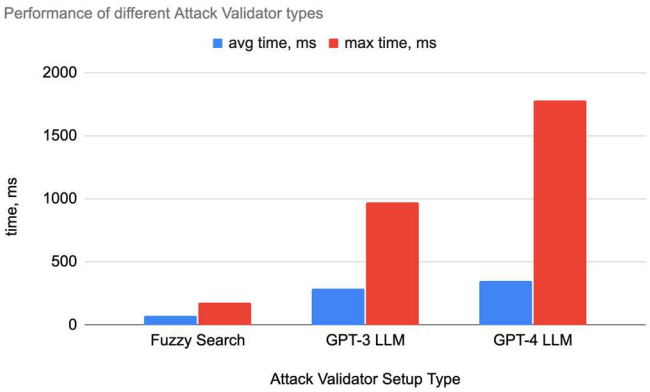


Fig. 2. Performance of tested different types of Attack validators.

## IV. Conclusions

This research provided a comprehensive analysis of cybersecurity vulnerabilities in systems using LM Interfaces, with a focus on prompt injection attacks as a significant threat. The study outlines a structural schema for systems incorporating LLMs, emphasizing multi-level zones where cybersecurity frameworks should be implemented. The Filter level, central to the system's security, was examined in detail, highlighting the benefits of integrating an additional security layer that preprocesses, wraps, and evaluates prompts using an LLM-based Attack Validator. The results clearly demonstrate that systems relying solely on core GPT models are highly vulnerable to various cyber-attacks, underscoring the essential need for specialized security filters. Empirical evidence shows that incorporating an Attack Validator, especially with the GPT-4 model, significantly enhances system resilience, with attack detection rates ranging from 76% to 97% depending on the type of prompt injection attack. Although the Fuzzy Search method offered advantages in response times, it lagged in accuracy and precision compared to GPT-based Attack Validators (versions 3 and 4). This highlights the complexity of cybersecurity in LM Interfaces and supports the need for advanced, AI-driven security solutions. The proposed filter security layer not only addresses current vulnerabilities but also positions the system proactively against emerging threats in the fast-evolving fields of AI and cybersecurity.

## References

[1] S. Arya, A. Bhaskar and K. Gupta, "Conversational Ai: A Treatise About Vying Chatbots," 2024 2nd International Conference on Disruptive Technologies (ICDT), Greater Noida, India, 2024, pp. 929-934, doi: 10.1109/ICDT61202.2024.10489545

[2] N. Kshetri, "Cybercrime and Privacy Threats of Large Language Models," in IT Professional, vol. 25, no. 3, pp. 9-13, May-June 2023, doi: 10.1109/MITP.2023.3275489

[3] A. Gobeo; C. Fowler; W.J. Buchanan, "GDPR and Cyber Security for Business Information Systems," in GDPR and Cyber Security for Business Information Systems, River Publishers, 2020, pp.1-19.

[4] I. Stankov and D. Gotseva, "An Overview of Security and Risk Management in Business Intelligence Systems," 2020 III International Conference on High Technology for Sustainable Development (HiTech), Sofia, Bulgaria, 2020, pp. 1-5, doi: 10.1109/HiTech51434.2020.9363990.

[5] K. Balamurugan, T. Sudalaimuthu, and V. Sherlin Solomi, "An Analysis of Various Cyber Threat Modeling," 2023 Third International Conference on Artificial Intelligence and Smart Energy (ICAIS), Coimbatore, India, 2023, pp. 426-429, doi: 10.1109/ICAIS56108.2023.10073771

[6] V.N. Nanisura Damanik and S.U. Sunaringtyas, "Secure Code Recommendation Based on Code Review Result Using OWASP Code Review Guide," 2020 International Workshop on Big Data and Information Security (IWBIS), Depok, Indonesia, 2020, pp. 153-158, doi: 10.1109/IWBIS50925.2020.9255559

[7] A. Serckumecka, I. Medeiros and A. Bessani, "Low-Cost Serverless SIEM in the Cloud," 2019 38th Symposium on Reliable Distributed Systems (SRDS), Lyon, France, 2019, pp. 381-3811, doi: 10.1109/SRDS47363.2019.00057

[8] O. I. Falowo, S. Popoola, J. Riep, V. A. Adewopo and J. Koch, "Threat Actors' Tenacity to Disrupt: Examination of Major Cybersecurity Incidents," in IEEE Access, vol. 10, pp. 134038-134051, 2022, doi: 10.1109/ACCESS.2022.3231847

[9] A. Athick, S. Banon, Getting Started with Elastic Stack 8.0: Run powerful and scalable data platforms to search, observe, and secure your organization, Packt Publishing, 2022, 474 p.

[10] O. Muliarevych The Cloud-Based Optimization for Automated Warehouse Design, 2023 12th IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS), Dortmund, Germany, 2023, pp. 311-316, doi: 10.1109/IDAACS.2023.8

Authorized licensed use limited to: Arizona State University. Downloaded on January 23,2026 at 15:59:11 UTC from IEEE Xplore. Restrictions apply.